

wpipe at the center of your pipelines

178: LinkedIn | 117k wpipe Users can't be wrong: Web Scraping is better with Checkpoints

The context

Scaling your data collection? Don't let crashes slow you down. Resilient: Resumes exactly where it left off.

Why engineers shift to wpipe

- Resilient: Resumes exactly where it left off.
- Lightweight: <50MB RAM.
- Transparent: Auto-generated Mermaid diagrams.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2, checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="pipeline.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]})

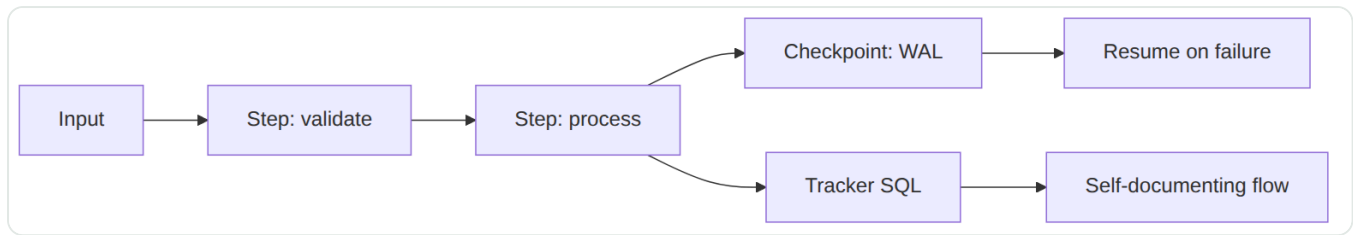
# Python-native orchestration: checkpoints, retries, self-documenting.
```

At a glance

Feature	wpipe	Standard Tools
Persistence	SQLite WAL	Hard to Implement
RAM	<50MB	Memory Leaks common

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **wpipe** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.